Technical Report: Temporal Knowledge Graph + X-FAMHA-GNN Cybersecurity Risk Assessment with Grounded LLM Assistant

Project type: Scoped academic reproduction + novel AI/LLM extension
Source paper: Bag, S., Sarkar, S., & Bose, I. (2025). *Enhancing cybersecurity risk assessment using temporal knowledge graph-based explainable decision support system. Decision Support Systems*, 198, 114526.
Hardware: NVIDIA RTX 2000 Ada Generation Laptop GPU (8 GB VRAM)
GitHub repository (code + docs): https://github.com/balajibrk/mba982-famha-cyber-risk
Purpose of this document: Provide a professor-reviewable account of (1) reproduction phases, (2) data sources and labeling, (3) AI model fidelity, and (4) LLM integration design—with emphasis on what is *claimed*, what is *proven*, and what is *scoped down*.

Resources (quick links)

Resource	Location
GitHub repository	https://github.com/balajibrk/mba982-famha-cyber-risk
Professor report (this document, Markdown)	`docs/PROFESSOR_REPORT.md`
Concise project summary	`docs/SUMMARY.md`
Labeling methodology	`docs/labeling_methodology.md`
Results table	`docs/results.csv`
Data manifest	`docs/data_manifest.csv`
KG statistics	`docs/kg_stats.csv`
Source paper (DOI)	https://doi.org/10.1016/j.dss.2025.114526
Streamlit demo entrypoint	`app/demo_app.py`
FAMHA implementation	`src/model/famha.py`
Counterfactual re-scoring	`src/assistant/counterfactual.py`
Grounded LLM narrative	`src/assistant/narrative.py`

Clone and browse:

```
git clone https://github.com/balajibrk/mba982-famha-cyber-risk.git
```

---

1. Executive summary

This project reproduces the core technical contribution of Bag et al. (2025)—a **Factor-Analysis-based Multi-Head Attention (FAMHA)** Graph Neural Network over **temporal cybersecurity knowledge**

graphs—and adds a novel **security-assistant layer** that the original paper does not include.

The assistant is deliberately *not* "an LLM that describes a chart." The language model only narrates **precomputed evidence**. The critical number it reports—the change in risk after a simulated remediation—is obtained by **re-running the trained GNN** on an edited knowledge graph. That separation is the central design claim of the LLM extension:

> Most "AI + LLM" demos have a chatbot describe a chart—talk, not proof. Here, the language layer reports a number the model actually recalculated after simulating the fix, not a number it made up. That is the difference between a security assistant that *sounds* smart and one that has *shown its work*.

Headline empirical results (leave-one-out CV, 18 companies):

Model	Accuracy	Macro-F1
X-FAMHA-GNN	**0.611**	**0.461**
GATConv baseline	0.444	0.329
Majority-class baseline	0.444	0.154

Across 5 random seeds, Mann–Whitney U (one-sided, X-FAMHA greater): **p = 0.0040** (F1 vs GAT), **p = 0.0037** (F1 vs majority). All below $\alpha = 0.05$.

Example counterfactual (model-computed, then narrated by LLM):

Company	Risk before (high+critical mass)	Risk after remediation simulation	Δ
Uber	1.000	0.041	–**95.9%**
Capital One	1.000	0.924	–**7.6%**

---

2. Problem statement and research positioning

2.1 What the paper asks

Organizations maintain cybersecurity *policies* (controls, standards, procedures) while also facing *attacks* (breaches, exploits, supply-chain compromises). The paper constructs **temporal knowledge graphs** linking policy and attack entities, then learns a graph classifier that assigns a **four-class vulnerability / policy-risk** label. The novel attention mechanism (FAMHA) uses **factor analysis** to choose the number of attention heads and to partition features, reducing parameters relative to vanilla multi-head attention while preserving predictive performance.

2.2 What this reproduction asks

1. Can FAMHA (Algorithm 1) be implemented faithfully enough that unit tests verify its mathematical claims (head-count response to eigenvalue spread; Θ_FAMHA < Θ_normal)?
2. On a *scoped* case-study set (~18 companies), does X-FAMHA-GNN beat a GATConv baseline and a majority-class baseline under leave-one-out CV, with a significance test?

3. Can interpretability (SHAP + FAMHA attention) surface entities that match known public breach narratives?
4. Can an LLM assistant layer be added **without** letting the LLM invent risk numbers—i.e., can impact be grounded in a real model re-score?

2.3 Novelty relative to the paper

Component	Paper	This project
Temporal KG + FAMHA GNN	Yes	Reproduced (faithful math, scoped data)
SHAP + attention heatmaps	Yes	Reproduced
Residual-risk / remediation impact	Static appendix-style tables	**Live re-score of trained GNN on edited KG**
Natural-language security assistant	No	**Local Ollama (Llama 3.1 8B) with evidence-only prompts**
Interactive demo	No	Streamlit app

---

3. Data sources

3.1 Honest framing

The paper uses large corpora (on the order of **190 train / 154 test** companies) with severity labels derived from sources such as **idtheftcenter.org** and **upguard.com**, and scrapes corporate policy / breach text. Those proprietary severity scales and the full scraped corpus were **not available** for this reproduction. Therefore this project uses a **curated seed corpus** and a **documented labeling heuristic**. This is a proof-of-concept case study, not a claim of parity with the paper's dataset size or label provenance.

3.2 Company set (N = 18)

Group	Count	Companies
Breached (public, documented incidents)	15	Uber, Capital One, Equifax, Target, Marriott, JPMorgan Chase, Citigroup, Wells Fargo, SolarWinds, Colonial Pipeline, Okta, LastPass, Progress MOVEit, Twitter, Yahoo
Comparatively clean (no major catastrophic customer-data breach of the type studied)	3	Apple, Cloudflare, Stripe

Metadata for each firm lives in `src/config.py` (sector, breach year, approximate records exposed, disclosure delay, recurrence, root-cause prose, policy areas, attack entities).

3.3 Text corpus (Phase 1)

Path	Content
`data/raw/policies/{company}.txt`	Authored cybersecurity-policy narrative grounded in realistic control language for that firm
`data/raw/attacks/{company}.txt`	Authored breach / threat narrative grounded in **documented public facts** about that firm's incident (or a "no major breach" statement for clean firms)

Every raw file carries a `# PROOF-OF-CONCEPT` provenance header stating that the text is a reconstruction for research reproduction, not a verbatim corporate document.

Why curated text? Live scraping of policy pages is brittle (robots.txt, layout churn, legal ambiguity) and unsuitable for a short solo timeline. Curated SVO-friendly prose also makes Open Information Extraction reliable without Stanford OpenIE / spaCy dependency parsers (see §7 environment constraints).

3.4 Labeling methodology (4 risk classes)

Labels are **not** taken from proprietary severity APIs. They are computed by an explicit heuristic documented in `docs/labeling_methodology.md`:

```
risk_score = factor_scope + factor_delay + factor_recurrence + factor_rootsev # 0..10
```

Factor	Basis	Range
Scope	log-bucket of records exposed	0–4
Disclosure delay	days to public disclosure	0–2
Recurrence	number of distinct major breaches	0–2
Root-cause severity	documented judgment for systemic / concealment / critical-infrastructure impact	0–2

Composite score	Class
0–1	`low_risk` (0)
2–4	`medium_risk` (1)
5–7	`high_risk` (2)
8–10	`critical_risk` (3)

Observed distribution: low 3 / medium 5 / high 8 / critical 2.

Limitations (stated for review): small N; skewed critical class; record-count proxy understates some operationally severe but low-record events (partly corrected by `root_severity`); public figures are approximate labeling proxies, not audited values.

3.5 Manifest and KG statistics

- `docs/data_manifest.csv` — per company × {policy, attack}: word counts, sentence counts, labels
- `docs/kg_stats.csv` — per-company graph sizes (nodes typically ~26–43, edges ~22–33), type counts, temporal attributes

---

4. Reproduction phases (pipeline)

End-to-end flow:

```
Curated policy + breach text → clean / tokenize / lemmatize → coreference resolution → SVO
open IE triples → cyber NER typing → verb clustering → 8 canonical relations → temporal
labels → per-company GraphML → node features (type ⊕ MiniLM ⊕ time) → FAMHA / X-FAMHA-GNN →
LOO training vs GATConv → SHAP + attention → counterfactual GNN re-score → grounded Ollama
narrative → Streamlit demo
```

Each phase has an automated validation script under `tests/validate_phase*.py`. All phases **PASS**.

Phase 0 — Setup and scoping

- Repository layout: `data/`, `src/{preprocess,kg,model,train,baselines,interpret,assistant}`, `app/`, `docs/`, `tests/`
- Central config: company list, 4-class scheme, canonical 8 relations, CUDA helpers
- Deliverable: `docs/labeling_methodology.md`

Phase 1 — Data collection and preprocessing

- Build curated corpus (`src/preprocess/build_corpus.py`)
- Cleaning (`src/preprocess/clean.py`): NLTK sentence/word tokenize → noise strip → SymSpell → **WordNetLemmatizer** (the lemmatizer cited by the paper)
- Outputs: cleaned sentence files + lemma streams + `docs/data_manifest.csv`
- Validation: non-empty policy **and** attack files for every company; no zero word counts

Phase 2 — Temporal knowledge graph construction

Step	Implementation	Notes vs paper
Coreference	Rule-based alias / “the company” → canonical name	Paper: neural coref
OIE	NLTK POS + NP-chunk SVO extractor	Paper: Stanford OpenIE
NER	Gazetteer + cyber lexicon → ORG / POLICY / ATTACK / ASSET / ENTITY	Paper: CyNER

Canonicalization	MiniLM embeddings + agglomerative clustering (cosine, τ=0.30) → map to 8 relations	Faithful to paper's clustering idea
Temporal	Breach year / publish proxy on nodes and edges	Temporal KG claim retained
Export	`{company}.graphml` + `docs/kg_stats.csv`	

Canonical relations (paper's compact set):
`implements`, `aligns-with`, `violates`, `mitigates`, `causes`, `impacts`, `reports`, `regulates`.

Observed: **526** raw triples → **117** unique verbs → **104** clusters → **all 8** relations used.

Phase 3 — FAMHA + X-FAMHA-GNN (core technical deliverable)

Node features: type one-hot (5) ⊕ MiniLM label embedding (384) ⊕ normalized year (1) → **d_in = 390**, then a learnable linear embedding to `d_model = 32`.

FAMHA (Algorithm 1), `src/model/famha.py`:

1. **Head count:** feature covariance eigenvalues; Kaiser rule (# eigenvalues > mean); unit test: concentrated spectrum → fewer heads than spread spectrum (e.g. 2 vs 12).
2. **Decomposition:** Principal Axis Factor Analysis assigns each of `d` dimensions to a factor → groups with `Σ len_i = d`.
3. **Attention:** per-head QKV; scores scaled by **√(d/2)**; softmax over neighbors (adjacency mask); **sigmoid** output; compose columns back to original order.
4. **Parameter theorem:** $\Theta\_FAMHA = 3 \Sigma len\_i^2 < 3 d^2 = \Theta\_normal$ when $h > 1$. Measured in trained stack: **1,638 vs 9,216 (~5.6× reduction)**.

Architecture: 3 FAMHA blocks + ELU + residual/FFN → global mean pool → 4-way softmax.

Validation: GPU forward/backward on real KG; `next(parameters).is_cuda == True`.

Phase 4 — Training and baselines

- Protocol: **leave-one-out CV** over 18 graphs (honest for small N; not the paper's 10-fold × 5-run on 190 firms)
- Metrics: accuracy, macro-F1, precision, recall, G-mean
- Baselines: **GATConv** (same features/protocol) + **majority-class**
- Significance: Mann–Whitney U over **5 seeds** of LOO macro-F1 / accuracy
- Artifacts: `docs/results.csv`, `artifacts/significance.json`, p-value heatmap

Result interpretation for review: X-FAMHA-GNN outperforms both baselines with $p < 0.05$. G-mean remains low because the critical class has only two examples under LOO—**expected and disclosed**, not papered over.

Phase 5 — Interpretability

- **SHAP KernelExplainer** over node-presence masks → top-entity bar charts
- Uber: credentials, AWS S3, related assets (matches public 2016 narrative)
- Capital One: cloud configurations, internal metadata (matches SSRF / cloud IAM narrative)
- **FAMHA attention heatmaps** (policy entities × attack entities)

Phase 6 — LLM assistant + demo

See §5 below. Streamlit app: company dropdown → risk class, SHAP, attention, counterfactual delta, grounded narrative.

---

5. AI and LLM integration (detailed)

5.1 Separation of concerns (design principle)

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ TRAINED
X-FAMHA-GNN ■ ■ • risk class probabilities ■ ■ • SHAP / attention evidence ■ ■ •
counterfactual re-score → risk_before, risk_after, Δ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ evidence
JSON only ▼ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
LLM (Ollama Llama 3.1 8B) OR deterministic template ■ ■ • narrates evidence ■ ■ • MUST use
exact counterfactual numbers ■ ■ • MUST NOT invent entities / statistics ■ ■ Output:
{one_line_verdict, why, fix, impact} ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

The LLM **never** scores risk. If Ollama is down, a template still emits the **same model numbers**—only prose quality changes. Impact remains model-true.

5.2 Counterfactual re-scoring (where "proof" comes from)

Implemented in `src/assistant/counterfactual.py`:

1. Forward pass on the current company graph → baseline high/critical risk mass.
2. **Simulate remediation:** neutralize attack-article weakness entities; inject a mitigating POLICY node (e.g. device MFA + least privilege) linked to the company.
3. Forward pass again on the edited graph → `risk_after`.
4. Report `delta` and `% change`.

This upgrades the paper's **static residual-risk table** idea into a **live, model-computed** what-if. That is the technical substance behind the "shown its work" claim.

5.3 Grounded narrative generation

Implemented in `src/assistant/narrative.py`:

- **Provider:** local **Ollama**, model `llama3.1:8b` (quantized / local; no cloud API dependency for the demo).
- **System rules:** evidence-only; risk-reduction number must match supplied delta; JSON-only response.
- **Schema:** `one_line_verdict`, `why`, `fix`, `impact`.
- **Post-check:** light scan for numeric tokens not present in evidence (`ungrounded_number_warnings`).

- **Views:** executive summary string + engineer-ticket JSON (title, severity, remediation, expected impact).

5.4 Real-time use cases (operational)

Use case	Trigger	Model role	LLM role
Analyst triage	Select company / Analyze	Class + SHAP + attention + Δ	Briefing in plain language
Exec one-pager	Same pipeline	Quantified risk change	Non-technical verdict
Engineering ticket	Same pipeline	Expected impact number	Ticket body from evidence
Remediation prioritization	Compare counterfactual Δ across firms	Rank by predicted impact	Explain why
Explainability QA	Inspect entities vs known breach	Attribution	Narrative for human check
Offline / LLM-down mode	Ollama unavailable	Unchanged Δ	Template prose

These are **interactive / on-demand** real-time analyses over the case-study KGs—not live SIEM stream monitoring. That scope boundary should be clear in any academic presentation.

5.5 Why this stands out among typical “AI + LLM” projects

Typical chatbot demo	This project
LLM estimates or invents “risk reduced by ~40%”	GNN recalculates risk; LLM quotes that number
Chart caption generation	Evidence package: SHAP + attention + edited-graph Δ
Cloud LLM as black box scorer	Local LLM as **narrator**; GNN as **reasoner**
Hard to audit	Evidence JSON + ungrounded-number check + template fallback

---

6. Results summary for evaluation

6.1 Classification (LOO)

Model	Accuracy	Macro-F1
X-FAMHA-GNN	0.611	0.461
GATConv	0.444	0.329
Majority	0.444	0.154

6.2 Robustness (5 seeds)

Model	Acc mean ± std	F1 mean
X-FAMHA-GNN	0.600 ± 0.042	0.495
GATConv	0.456 ± 0.074	0.340

Mann–Whitney U p-values (X-FAMHA greater): F1 vs GAT **0.0040**; Acc vs GAT **0.0096**; F1 vs majority **0.0037**.

6.3 FAMHA efficiency

Trained attention footprint **1,638** parameters vs **9,216** vanilla multi-head equivalent (same `d`), consistent with Theorem 3.1.

6.4 Interpretability sanity

Top SHAP entities for Uber and Capital One align with publicly documented breach mechanisms (credentials/S3; cloud config / metadata).

6.5 Counterfactual + LLM

Non-zero model-computed deltas; Ollama narratives validated for schema conformance and absence of ungrounded numbers in the Phase 6 validation suite.

---

7. Environment, fidelity, and deviations

7.1 Stack

- Python **3.12** via `uv` (system Python 3.14 lacked reliable ML wheels)
- PyTorch **2.6 + CUDA 12.4**, Torch Geometric, sentence-transformers (`all-MiniLM-L6-v2`), scikit-learn, SHAP, NLTK, Streamlit, Ollama

7.2 Forced deviations (Windows Smart App Control)

Unsigned compiled DLLs were blocked on the build machine:

1. **spaCy** → **NLTK** for tokenize / POS / lemmatize (WordNetLemmatizer matches the paper's cited tool more closely than spaCy).
2. **numba/llvmlite** → **no-op shim** so SHAP runs in pure-Python mode (correctness preserved; speed only).

These are documented in `README.md` and `requirements.txt` comments—not hidden.

7.3 Scope-down vs paper (table for reviewers)

Aspect	Paper	This repo	Reason
Companies	~190 / 154	18	Solo timeline; PoC corpus
Labels	idtheftcenter / upguard	Documented heuristic	No proprietary API

Text	Scraped	Curated, fact-grounded	Scraping fragility / time
Baselines	10 SOTA	GATConv + majority	Time-box; foundational GAT baseline
Benchmarks	4 datasets	Case study only	Focus on FAMHA + assistant
Coref / OIE / NER	Neural / OpenIE / CyNER	Rule / NLTK / gazetteer	SAC + dependency constraints
FAMHA math	Original	Faithful reimplementation	Core claim retained

---

8. Limitations and threats to validity

1. **External validity:** N = 18 cannot match the paper's statistical power. Results are illustrative of mechanism + pipeline, not a definitive industry ranking.
2. **Construct validity of labels:** Heuristic labels approximate severity; they are transparent but not identical to the paper's sources.
3. **Corpus construct:** Authored text is SVO-friendly and fact-grounded but is not scraped ground truth; KG structure partly reflects authoring choices.
4. **Class imbalance:** Only two critical firms → LOO struggles on that class; G-mean reflects this honestly.
5. **LLM residual risk:** Prompting + numeric checks reduce hallucination but do not eliminate all linguistic drift; the **numeric impact** remains trustworthy because it is model-sourced.
6. **Counterfactual semantics:** Remediation is a structured graph edit (mask attack entities + inject MFA control), not a full SOC playbook simulation.

---

9. Reproducibility

Primary instructions: repository `README.md` and `docs/SUMMARY.md`.

```
uv venv --python 3.12 .venv uv pip install --python .venv\Scripts\python.exe torch --index-url
https://download.pytorch.org/whl/cu124 uv pip install --python .venv\Scripts\python.exe -r
requirements.txt # NLTK data downloads as documented in README .venv\Scripts\python.exe -m
src.preprocess.build_corpus .venv\Scripts\python.exe -m src.preprocess.clean
.venv\Scripts\python.exe -m src.kg.build_graph .venv\Scripts\python.exe -m src.model.features
.venv\Scripts\python.exe -m src.train.train_case_study .venv\Scripts\python.exe -m
src.baselines.run_baselines .venv\Scripts\python.exe -m src.train.significance
.venv\Scripts\python.exe -m src.interpret.shap_explain .venv\Scripts\python.exe -m
src.assistant.pipeline uber capital_one .venv\Scripts\streamlit run app\demo_app.py
```

Validation: `python -m tests.validate_phase1` … `validate_phase6`.

---

10. Repository map (for reviewers)

Path	Role

`src/model/famha.py`	Algorithm 1 (FAMHA)
`src/model/xfamha_gnn.py`	Full classifier
`src/kg/`	Coref, OIE, NER, canonicalize, temporal, GraphML build
`src/train/`	LOO training, significance
`src/baselines/run_baselines.py`	GATConv
`src/interpret/`	SHAP + attention
`src/assistant/`	Counterfactual + Ollama narrative + pipeline
`app/demo_app.py`	Streamlit UI
`docs/labeling_methodology.md`	Label rules
`docs/SUMMARY.md`	Concise project summary
`docs/PROFESSOR_REPORT.md`	This document
`tests/validate_phase*.py`	Phase gates

---

11. Conclusion

This work delivers a **complete, phase-gated reproduction** of the FAMHA temporal-KG cybersecurity risk pipeline on a scoped but transparent case study, with **faithful Algorithm 1 mathematics**, **competitive LOO results against GAT**, and a **novel LLM assistant** whose impact claims are **bound to live GNN counterfactual re-scoring**.

For academic review, the recommended reading order is:

1. This report (§3 data honesty, §5 LLM proof claim, §8 limitations)
2. `docs/labeling_methodology.md`
3. `src/model/famha.py` + `src/assistant/counterfactual.py` + `src/assistant/narrative.py`
4. `docs/results.csv` and Phase validation outputs

The contribution to “AI + LLM” discourse is methodological: **language is for explanation and packaging; prediction and impact measurement stay with the graph model.** That is what makes the assistant auditable rather than merely articulate.

---

12. References and resources

Primary literature

1. Bag, S., Sarkar, S., & Bose, I. (2025). Enhancing cybersecurity risk assessment using temporal knowledge graph-based explainable decision support system. *Decision Support Systems*, *198*, 114526. https://doi.org/10.1016/j.dss.2025.114526

Software and model resources used in this reproduction

Component	Resource
Graph learning	PyTorch Geometric — https://pytorch-geometric.readthedocs.io/
Deep learning	PyTorch (CUDA) — https://pytorch.org/
Sentence embeddings	`sentence-transformers` / `all-MiniLM-L6-v2` — https://www.sbert.net/
Factor analysis	scikit-learn `FactorAnalysis` — https://scikit-learn.org/
Interpretability	SHAP (KernelExplainer) — https://shap.readthedocs.io/
Local LLM runtime	Ollama — https://ollama.com/
LLM weights	Meta Llama 3.1 8B via `ollama pull llama3.1:8b`
Demo UI	Streamlit — https://streamlit.io/
NLP (lemmatization)	NLTK WordNetLemmatizer — https://www.nltk.org/

Project deliverables

Deliverable	URL / path
Public GitHub repository	**https://github.com/balajibrk/mba982-famha-cyber-risk**
This report (Markdown)	https://github.com/balajibrk/mba982-famha-cyber-risk/blob/main/docs/PR
Summary	https://github.com/balajibrk/mba982-famha-cyber-risk/blob/main/docs/SU
Labeling rules	https://github.com/balajibrk/mba982-famha-cyber-risk/blob/main/docs/lab
README / setup	https://github.com/balajibrk/mba982-famha-cyber-risk/blob/main/READM

Document version: 1.1 — prepared for professor review. Companion code: https://github.com/balajibrk/mba982-famha-cyber-risk